

DATA 260 Homework 5

Student: Vrishin KP

Project: Package Vulnerability Reporting Platform

Technologies: FastAPI, SQLAlchemy, MySQL, React, Redux Toolkit, Axios, and Vite

1. Objective

The goal of this homework was to improve the existing vulnerability reporting application by connecting the React frontend to Redux Toolkit. Earlier versions handled some report information directly inside individual pages. In this homework, the main report state and CRUD operations were moved into Redux so the application has a more organized data flow.

This work also connected the frontend and backend more clearly. React collects the user input, Redux manages the application state, Axios sends the request, and FastAPI performs the database operation. The new HW5 fields, including package ID and affected versions count, were also added to the frontend and backend workflow.

2. Implemented Features

I implemented and tested the following features:

- Connected report loading to the Redux store.
- Added Redux thunks for loading, creating, updating, and deleting reports.
- Added package ID support to the create and update forms.
- Added affected versions count support.
- Updated the backend to save affected-version counts.
- Updated the backend timestamp when a report is modified.
- Connected the delete screen to the Redux delete thunk.
- Verified the production frontend build using Vite.

During testing, I found that the affected versions count was still being saved as zero after an update. The backend update function was corrected so that it saves the submitted value and updates the timestamp. I then repeated the browser test and confirmed that the value was saved correctly.

3. Redux Data Flow

Redux is used as the central location for the report list and report operations. When a user performs an action, the page calls a handler in `App.jsx`. The handler dispatches a Redux thunk. The thunk calls the Axios API function, and the response updates the Redux state.

This approach keeps the page components focused on displaying forms and collecting input. It also prevents different pages from keeping separate copies of the same report data.

Operation	Redux action	Backend operation
Load reports	<code>fetchReports()</code>	GET vulnerability reports
Create report	<code>addReport(payload)</code>	POST a new vulnerability report
Update report	<code>editReport({ id, payload })</code>	PUT updated report data
Delete report	<code>removeReport(id)</code>	DELETE the selected report

The main request path is: page component → `App.jsx` → Redux thunk → Axios API function → FastAPI endpoint → database.

4. Verification Results

I tested the application through the browser using the local React frontend and FastAPI backend. The tests were completed after logging in with the demo account.

Test	Result	Evidence
User login and report loading	Passed	The Home page displayed vulnerability reports after login.
Update report	Passed	The affected versions count was changed to 5 and saved correctly.
Create report	Passed	A new report with a unique vulnerability ID appeared in the list.

Delete report	Passed	The deleted report no longer appeared on the Home page.
Production build	Passed	Vite completed successfully with 95 modules transformed.

5. Test Evidence

The following screenshots document the browser tests. Passwords and authentication tokens were not included in the evidence.

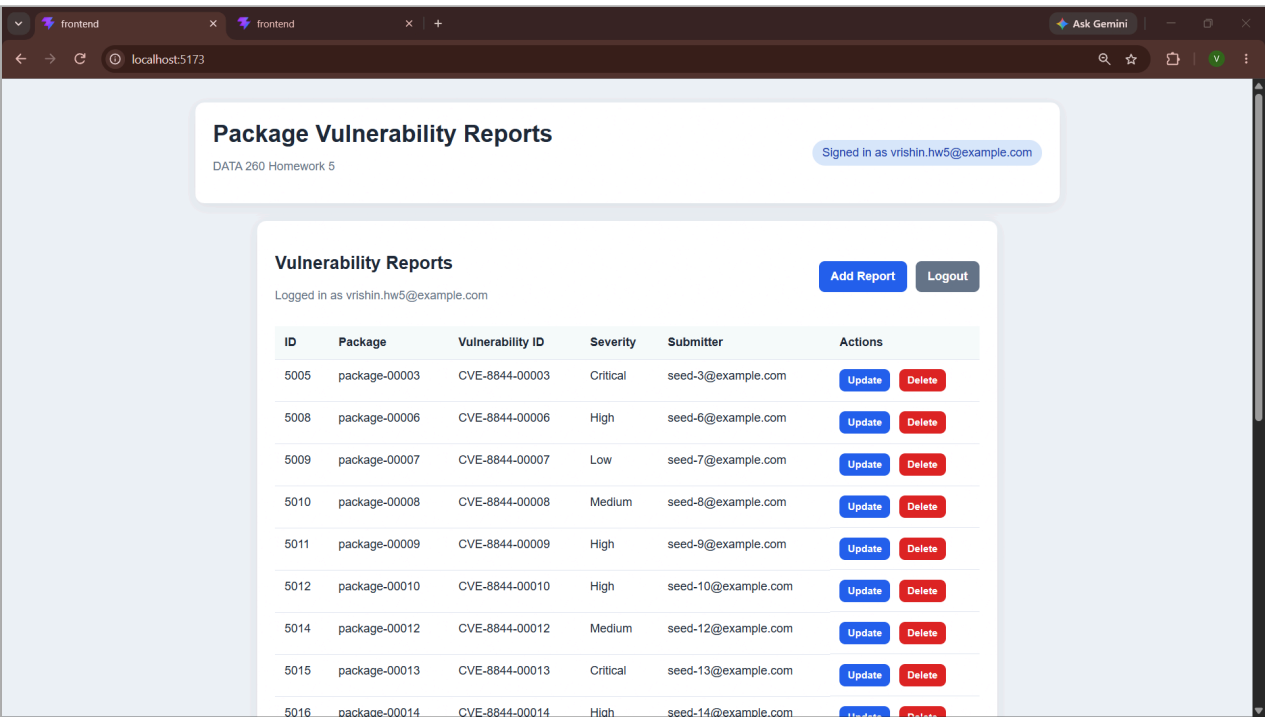


Figure 1. Authenticated Home page showing vulnerability reports.

Update Vulnerability Report

Package name
package-00006

Package ID
6

Vulnerability ID
CVE-8844-00006

Submitter email
seed-6@example.com

Vulnerability description
Deterministic benchmark vulnerability description for record 6.

Severity
High

Affected versions count
5

☒ I accept the terms and conditions.

Update Vulnerability Report

Figure 2. Update form showing the affected versions count set to 5.

Package Vulnerability Reports

DATA 260 Homework 5

Signed in as vishin.hw5@example.com

Vulnerability Reports

Logged in as vishin.hw5@example.com

Add Report Logout

ID	Package	Vulnerability ID	Severity	Submitter	Actions
5005	package-00003	CVE-2026-8844-TEST-003	Critical	seed-3@example.com	Update Delete
5008	package-00006	CVE-8844-00006	High	seed-6@example.com	Update Delete
5009	package-00007	CVE-8844-00007	Low	seed-7@example.com	Update Delete
5010	package-00008	CVE-8844-00008	Medium	seed-8@example.com	Update Delete
5011	package-00009	CVE-8844-00009	High	seed-9@example.com	Update Delete
5012	package-00010	CVE-8844-00010	High	seed-10@example.com	Update Delete
5014	package-00012	CVE-8844-00012	Medium	seed-12@example.com	Update Delete
5015	package-00013	CVE-8844-00013	Critical	seed-13@example.com	Update Delete
5016	package-00014	CVE-8844-00014	High	seed-14@example.com	Update Delete

Figure 3. Home page showing the newly created vulnerability report.

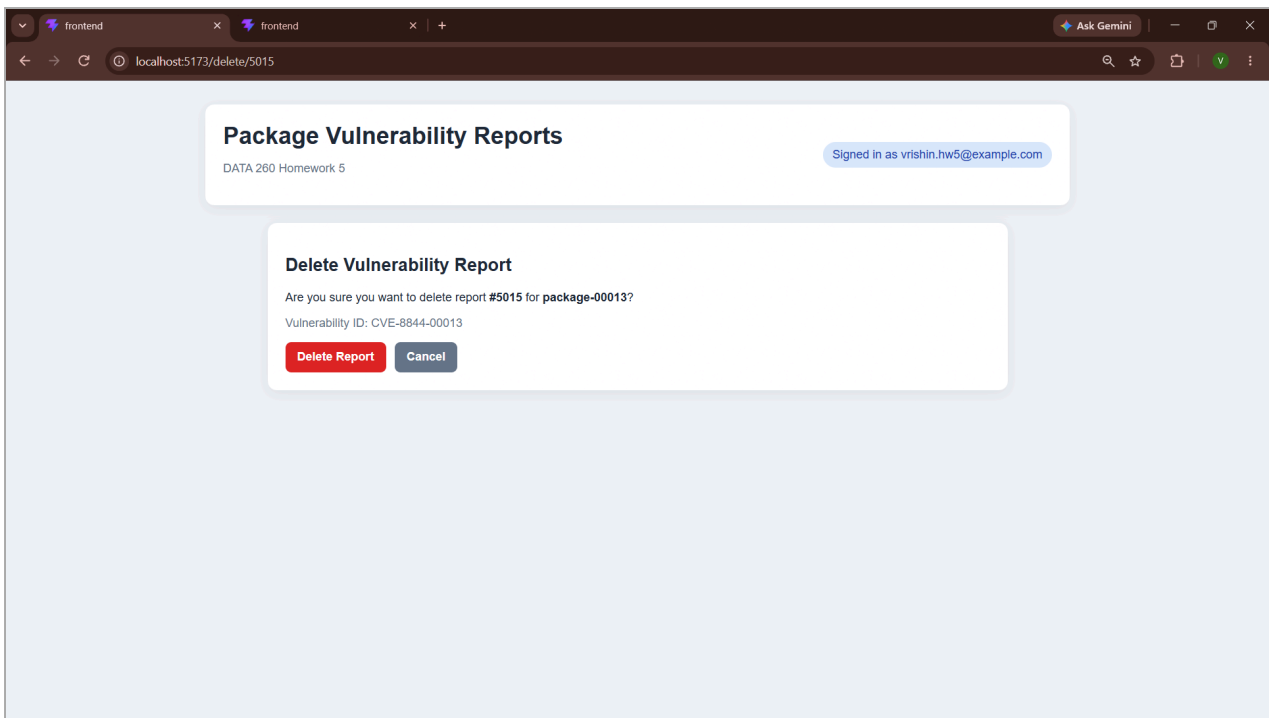


Figure 4. Delete confirmation page before removing the report.

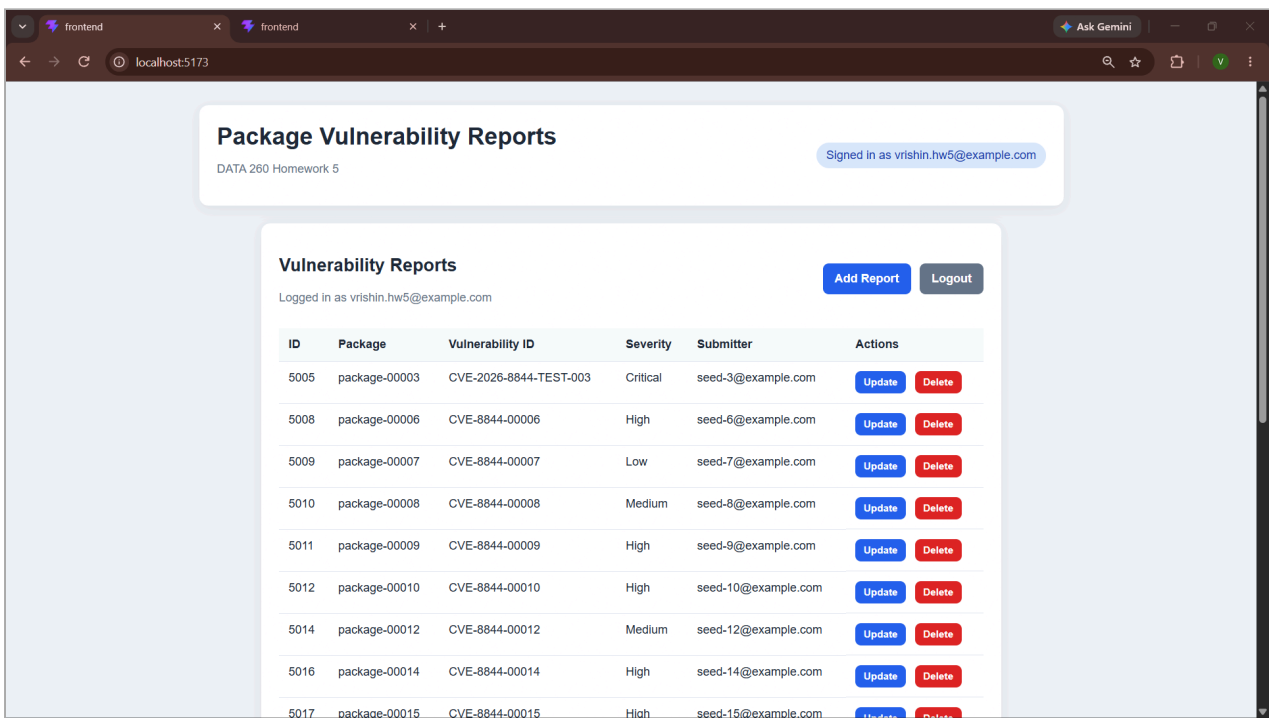


Figure 5. Home page after deletion, showing that the report was removed.

```

PS C:\Projects\data260-8844> cd "C:\Projects\data260-8844\frontend"
PS C:\Projects\data260-8844\frontend> npm run build

> frontend@0.0.0 build
> vite build

vite v8.3.1 building client environment for production...
/ 95 modules transformed.
computing gzip size...
dist/index.html          0.45 kB | gzip: 0.29 kB
dist/assets/index-wprW19rh.css 1.95 kB | gzip: 0.85 kB
dist/assets/index-B1Vd4o0J.js 349.11 kB | gzip: 112.62 kB

/ built in 240ms

```

Figure 6. Successful production build using `npm run build`.

6. Important Project Files

These files were central to the HW5 implementation:

- `frontend/src/App.jsx` - connects pages to Redux actions and selectors.
- `frontend/src/features/reports/reportsSlice.js` - stores report state and defines Redux thunks.
- `frontend/src/store/store.js` - configures the Redux store.
- `frontend/src/pages/CreateRecord.jsx` - creates reports with the new HW5 fields.
- `frontend/src/pages/UpdateRecord.jsx` - loads and updates existing reports.
- `frontend/src/pages/DeleteRecord.jsx` - confirms and deletes reports.
- `code/web_application/crud_hw4.py` - performs database CRUD operations.
- `code/web_application/schemas.py` - validates API request and response data.

7. Challenges and Fixes

The main issue during testing was that the affected versions count displayed as zero after an update. The frontend was sending the value, but the backend update function was not assigning it to the database record.

I corrected the backend update logic, restarted FastAPI, and repeated the test. The value 5 was then saved and displayed correctly after reopening the report.

I also corrected an accidental nested frontend folder that was created while editing the delete page. The corrected file was moved into the actual `frontend/src/pages` directory, and the frontend build was run again successfully.

8. Submission Information

The final homework implementation was committed and pushed to the repository using the HW5 development branch. An annotated Git tag was created so the complete submission can be identified easily.

Repository	Viki2183/data260-8844
Branch	hw5-development
Git tag	hw5
Commit	a28a0ba
Report folder	reports/hw05
PDF file	KunnathamParambath_HW5.pdf

9. Conclusion

Homework 5 was completed by integrating Redux Toolkit into the existing vulnerability reporting application. The report list and CRUD operations now follow a consistent Redux-based data flow.

The browser tests confirmed that a user can log in, view reports, create a report, update an existing report, save the affected versions count, and delete a report. The final Vite production build also completed successfully, confirming that the frontend can be bundled for deployment.